

Step 1 — A Script That Sends One Message and Prints the Reply — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# chat_client.py
from openai import OpenAI

def create_client(api_key):
    return OpenAI(api_key=api_key)

def send_message(client, message):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": message}],
    )
    return response.choices[0].message.content
```

```
# main.py
from config import load_config
from chat_client import create_client, send_message

config = load_config()
client = create_client(config.openai_api_key)

reply = send_message(client, "What's 2+2?")
print(reply)
```

This works. It's missing type hints and doesn't use the logger yet — both fine for a first pass that just proves the connection works, but worth adding once you know the checks pass. It's also missing any check that the key was valid *before* this call — if it's wrong, this fails right here, mixed in with the real test.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with logging

```
# chat_client.py
from openai import OpenAI
from config import Config

def create_client(config: Config) -> OpenAI:
    return OpenAI(api_key=config.openai_api_key)

def send_message(client: OpenAI, message: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": message}],
    )
    return response.choices[0].message.content
```

```
# main.py
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, send_message

config = load_config()
logger = get_logger(__name__)
client = create_client(config)

logger.info("Sending test message to the model")
reply = send_message(client, "What's 2+2?")
logger.info("Received reply: %s", reply)
print(reply)
```

Difference from Basic: full type hints on both functions, so any later file that imports `send_message` knows exactly what it needs and what it gets back. `create_client()` takes the whole `Config` object instead of a bare string — this matters later, once `Config` grows more fields (like a model name or timeout) that `create_client()` might also want, without changing its signature again. And `main.py` logs what it's doing instead of only printing — once this script is one

piece of a bigger app (Step 4's router), those log lines are how you'll trace what happened. This version still only discovers a bad key when `send_message()` itself is called — that's what Advanced fixes.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a free startup check with `models.list()`

```
from openai import OpenAI, AuthenticationError
from config import Config

def create_client(config: Config) -> OpenAI:
    return OpenAI(api_key=config.openai_api_key)

def check_api_key_at_startup(client: OpenAI) -> None:
    try:
        client.models.list()
    except AuthenticationError:
        raise SystemExit("Your OPENAI_API_KEY is invalid. Fix your .env before co

def send_message(client: OpenAI, message: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": message}],
    )
    return response.choices[0].message.content
```

```
# main.py
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, send_message, check_api_key_at_startup

config = load_config()
logger = get_logger(__name__)
client = create_client(config)
check_api_key_at_startup(client)

reply = send_message(client, "What's 2+2?")
print(reply)
```

Expected behavior: with a valid key, `check_api_key_at_startup()` returns silently and the script proceeds exactly as before. With an invalid key, it exits immediately with a clear one-line message — `send_message()` is never even called. `models.list()` costs no completion tokens, so this check is essentially free to run every time the app starts.

Approach 2 — a stricter startup check with a 1-token completion

```
def check_api_key_at_startup(client: OpenAI) -> None:
    try:
        client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "ping"}],
            max_tokens=1,
        )
    except AuthenticationError:
        raise SystemExit("Your OPENAI_API_KEY is invalid. Fix your .env before co
```

Expected behavior: the same as Approach 1 — fails fast with a clear message on a bad key. The difference is what it actually tests. `models.list()` only proves the key authenticates. This call proves the exact endpoint your app depends on (`chat.completions.create`) is reachable and working, at the cost of a sliver of a token.

Approach 3 — what the same idea looks like as a real service's health check

This project is a terminal script, not a web server, so there's nothing to wire this into here. But this is the shape the same idea takes once an app like this one becomes a real, always-on service — exactly where Project 5 in this curriculum is headed:

```
# sketch only – not part of this terminal project
from fastapi import FastAPI

app = FastAPI()

@app.get("/healthz")
def healthz():
    # liveness: is the process even running? Checked constantly.
    return {"status": "ok"}

@app.get("/readyz")
def readyz():
    # readiness: can it actually do its job right now?
    try:
        client.models.list()
        return {"status": "ready"}
```

```
except AuthenticationError:
    return {"status": "not ready", "reason": "invalid API key"}, 503
```

An orchestrator (like Kubernetes) polls `/healthz` every few seconds to decide whether to restart the process. It polls `/readyz` to decide whether to send the process real traffic. This project's `check_api_key_at_startup()` is the readiness half of that pattern, just simplified to "check once at boot" — a short terminal session doesn't need a repeating schedule.

Difference from Intermediate, and between these 3 Advanced approaches: Intermediate only discovers a bad key by accident, whenever `send_message()` first happens to be called. All three Advanced approaches check on purpose, before the user types anything, and fail with one clear sentence instead of a raw SDK traceback. Approach 1 is essentially free and enough to catch a bad key. Approach 2 costs a token but proves the exact call path your app actually uses. Approach 3 isn't code this terminal project runs. It shows what the same check looks like once it has to run again and again, unattended, instead of once at boot.

Which one should you actually write? Approach 2 — the 1-token completion — is what belongs in this project. It's exactly what Step 3's solution wires in as `check_api_key_at_startup()`, because it proves the specific call your app depends on actually works, not just that the key authenticates in general. Approach 1 (`models.list()`) is a good, slightly cheaper alternative if you only care about catching an invalid key. Approach 3 is worth understanding now, even though you won't write it for this project, so the readiness/liveness split feels familiar once this curriculum's later projects turn work like this into a real running service.